

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验 |LAB4

Pthreads 并行方程求解及蒙特卡洛

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 吴迪、刘学正

起 始 日 期: 2026 年 5 月 5 日

结 束 日 期: 2026 年 5 月 8 日

实 验 地 点: B202

目录

1 实验简述 **2**
1.1 实验内容 . . . 2
1.2 项目结构 . . . 2
1.3 环境说明 . . . 2

2 分析设计 **3**
2.1 方程求解的任务依赖设计 . . . 3
2.1.1 任务分解与 DAG . . . 3
2.1.2 条件变量同步机制 . . . 3
2.2 蒙特卡洛的无锁并发设计 . . . 4
2.2.1 线程安全随机数生成 . . . 4
2.2.2 局部累加避免锁竞争 . . . 4

3 核心代码 **4**
3.1 方程求解的条件变量同步实现 . . . 4
3.1.1 共享数据结构设计 . . . 4
3.1.2 Thread 1：计算判别式 Δ . . . 5
3.1.3 Thread 2：等待 Δ 并计算 $\sqrt{\Delta}$ . . . 6
3.1.4 Thread 3/4：并行计算 x 和 x . . . 6
3.2 蒙特卡洛的线程安全高性能设计 . . . 7
3.2.1 线程数据结构：独立种子设计 . . . 7
3.2.2 线程函数：无锁并行计算 . . . 8
3.2.3 主线程：种子初始化与结果汇总 . . . 8

4 结果分析 **9**
4.1 方程求解测试结果 . . . 9
4.2 蒙特卡洛测试结果 . . . 10

5 实验总结 **11**
5.1 技术收获 . . . 11
5.2 问题与反思 . . . 12

1 实验简述

1.1 实验内容

本实验主要包含两个并行计算任务：

任务一：一元二次方程并行求解。使用 Pthreads 多线程和条件变量（`pthread_cond_t`）实现一元二次方程 $ax^2 + bx + c = 0$ 的并行求解。要求将计算过程拆分为多个步骤（计算 Δ、计算 $\sqrt{\Delta}$、计算根），并通过条件变量实现线程间的同步与依赖等待。

任务二：蒙特卡洛方法并行计算 Pi。使用 Pthreads 多线程实现蒙特卡洛方法估算圆周率 π。要求使用线程安全的随机数生成器（`rand_r`），每个线程拥有独立的种子，避免全局锁竞争，实现高性能并行计算。

1.2 项目结构

本项目采用以下目录结构组织代码和文档：

```
lab4/
  lab4.tex                # 实验报告LaTeX源码
  results.txt             # 测试结果数据
  img/
     SYSULogo.png         # 校徽
     mc_speedup.png       # 加速比折线图
     mc_efficiency.png    # 并行效率图
     run_eq.png           # 方程求解运行截图
     run_mc.png           # 蒙特卡洛运行截图
  code/
     requirements.txt     # Python依赖清单
     eq_solver.c          # 方程求解程序
     mc_pi.c              # 蒙特卡洛程序
     run_tests.sh         # 自动化测试脚本
     plot_results.py      # 数据可视化脚本
```

1.3 环境说明

实验在 WSL2 Ubuntu 22.04 环境下进行，使用 GCC 编译器和 Make 工具。Python 可视化部分使用虚拟环境（venv）隔离依赖，确保环境一致性。

环境配置步骤：

1. 创建 Python 虚拟环境：`python3 -m venv venv`
2. 激活虚拟环境：`source venv/bin/activate`
3. 安装依赖：`pip install -r code/requirements.txt`
4. 编译 C 程序：`gcc -O2 -pthread source.c -o output -lm`

2 分析设计

2.1 方程求解的任务依赖设计

2.1.1 任务分解与 DAG

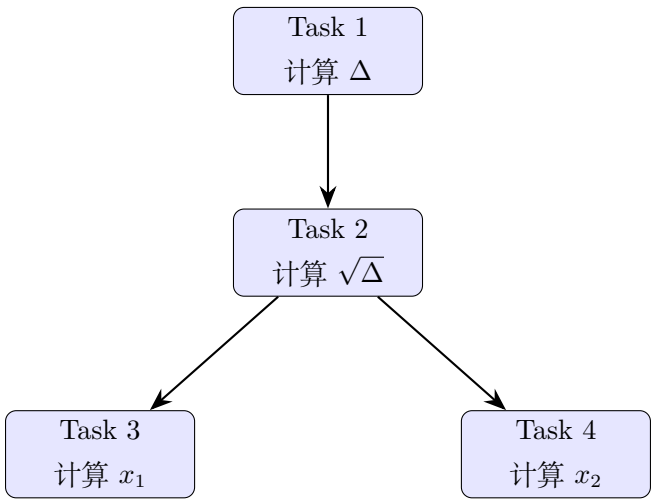
一元二次方程求解公式为：

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{1}$$

将计算过程分解为以下子任务，形成有向无环图（DAG）：

- **Task 1**：计算判别式 $\Delta = b^2 - 4ac$
- **Task 2**：计算 $\sqrt{\Delta}$（依赖 Task 1，需等待 Δ 计算完成）
- **Task 3**：计算 $x_1 = (-b + \sqrt{\Delta})/(2a)$（依赖 Task 2）
- **Task 4**：计算 $x_2 = (-b - \sqrt{\Delta})/(2a)$（依赖 Task 2）

任务依赖关系如图所示：



2.1.2 条件变量同步机制

使用 `pthread_cond_t` 实现线程间的条件等待：

1. **条件变量 `cond_delta`**：用于通知 Δ 计算完成
2. **条件变量 `cond_sqrt_delta`**：用于通知 $\sqrt{\Delta}$ 计算完成
3. **条件变量 `cond_roots`**：用于通知根计算完成

核心同步逻辑：

- Thread 2 在计算 $\sqrt{\Delta}$ 前，必须通过 `pthread_cond_wait` 等待 `delta_ready` 标志
- Thread 1 完成 Δ 计算后，通过 `pthread_cond_signal` 唤醒 Thread 2
- Thread 3 和 Thread 4 同时等待 $\sqrt{\Delta}$ 完成，使用 `pthread_cond_broadcast` 唤醒两个线程

2.2 蒙特卡洛的无锁并发设计

2.2.1 线程安全随机数生成

标准 `rand()` 函数使用全局状态，多线程调用需要加锁，会导致严重的性能瓶颈。本实验采用 `rand_r` 函数，每个线程拥有独立的种子：

- 每个线程初始化时设置独立种子：`seed = time(NULL) + thread_id * 12345`
- 使用 `rand_r(&seed)` 生成随机数，无需全局锁
- 不同种子确保各线程生成不同的随机序列

2.2.2 局部累加避免锁竞争

采用”局部累加 + 全局汇总”策略：

1. 每个线程维护局部计数器 `local_count`
2. 线程内部循环中只更新局部变量，无需同步
3. 线程完成后，通过互斥锁一次性将局部结果汇总到全局变量

这种设计将锁操作从 $O(N)$ 次降低到 $O(num_threads)$ 次，大幅减少锁竞争开销。

3 核心代码

本节详细展示两个程序的核心实现代码，并结合设计思路、优化策略和创新点进行深入讲解。

3.1 方程求解的条件变量同步实现

3.1.1 共享数据结构设计

在多线程编程中，共享数据结构的设计至关重要。本程序采用结构体封装所有共享变量，实现数据的集中管理和同步原语的统一初始化。

```
typedef struct {
    double a, b, c;          // 方程系数（输入参数）
    double delta;            // Δ = b² - 4ac（中间结果）
    double sqrt_delta;       // √Δ（中间结果）
    double x1, x2;           // 方程的根（最终结果）
    int has_real_roots;      // 是否有实根（状态标志）

    // 同步原语
    pthread_mutex_t mutex;             // 互斥锁保护共享数据
    pthread_cond_t cond_delta;         // 条件变量：Δ计算完成
    pthread_cond_t cond_sqrt_delta;    // 条件变量：√Δ计算完成
    pthread_cond_t cond_roots;         // 条件变量：根计算完成

    int delta_ready;         // Δ已计算标志
    int sqrt_delta_ready;    // √Δ已计算标志
    int x1_ready;            // x1已计算标志
    int x2_ready;            // x2已计算标志
} SharedData;
```

Listing 1: 共享数据与同步原语定义

代码设计要点：

1. **数据封装**：将所有共享变量封装在结构体中，避免全局变量散落，便于管理和传递。线程函数只需接收一个指针参数即可访问所有数据。
2. **状态标志位**：使用 `delta_ready` 等布尔标志记录计算进度。这是条件变量正确使用的关键——必须配合状态变量，防止虚假唤醒（spurious wakeup）导致逻辑错误。
3. **多条件变量设计**：设计了三个独立的条件变量，分别对应不同的计算阶段。这种细粒度设计避免了不必要的唤醒：计算 x_1 的线程只需等待 $\sqrt{\Delta}$ 完成，无需关心 x_2 的状态。

3.1.2 Thread 1：计算判别式 Δ

Thread 1 是流水线的起点，无需等待任何前置条件，直接计算 Δ 并唤醒下游线程。

```c
void* compute_delta(void* arg) {
    SharedData* data = (SharedData*)arg;

    pthread_mutex_lock(&data->mutex);

    // 计算Δ = b² - 4ac（无需等待，直接计算）
    data->delta = data->b * data->b - 4 * data->a * data->c;

    // 判断是否有实根，为后续线程提供决策依据
    if (data->delta >= 0) {
        data->has_real_roots = 1;
    } else {
        data->has_real_roots = 0;
    }

    // 设置完成标志
    data->delta_ready = 1;

    // 唤醒等待Δ的线程（Thread 2）
    pthread_cond_signal(&data->cond_delta);

    pthread_mutex_unlock(&data->mutex);
    return NULL;
}
```

Listing 2: Thread 1：计算 Δ 并唤醒下游

代码设计要点：

- **锁的必要性**：虽然 Thread 1 是第一个执行的线程，但仍需加锁。因为 `delta_ready` 标志会被其他线程读取，必须保证写入操作的原子性和可见性。
- **决策信息传递**：`has_real_roots` 标志是重要的决策信息。Thread 2 根据此标志决定是否计算 $\sqrt{\Delta}$，Thread 3 和 Thread 4 根据此标志决定输出实根还是复根。
- **signal vs broadcast**：此处使用 `pthread_cond_signal` 而非 `broadcast`，因为只有 Thread 2 在等待 `cond_delta`。精确唤醒减少不必要的线程调度开销。

3.1.3 Thread 2：等待 Δ 并计算 √Δ

Thread 2 展示了条件变量的经典使用模式——在 while 循环中等待条件成立。

```
void* compute_sqrt_delta(void* arg) {
    SharedData* data = (SharedData*)arg;

    pthread_mutex_lock(&data->mutex);

    // 【关键】在while循环中等待条件，防止虚假唤醒
    while (!data->delta_ready) {
        pthread_cond_wait(&data->cond_delta, &data->mutex);
    }

    // 根据前置决策信息选择计算路径
    if (data->has_real_roots) {
        data->sqrt_delta = sqrt(data->delta);
    } else {
        data->sqrt_delta = 0;  // 无实根时置零
    }

    data->sqrt_delta_ready = 1;

    // 【关键】使用broadcast唤醒两个并行线程（x1和x2）
    pthread_cond_broadcast(&data->cond_sqrt_delta);

    pthread_mutex_unlock(&data->mutex);
    return NULL;
}
```

Listing 3: Thread 2：条件变量等待与唤醒广播

代码设计要点：

1. **while 循环等待**：这是条件变量使用的黄金法则。`pthread_cond_wait` 可能因系统信号或调度原因返回，但条件未必成立。必须在 while 循环中重新检查条件：
   - 错误写法：`if (!ready) pthread_cond_wait(...);`
   - 正确写法：`while (!ready) pthread_cond_wait(...);`
2. **cond_wait 的原子性**：`pthread_cond_wait` 原子地释放互斥锁并进入等待状态。被唤醒时，它重新获取锁后才返回。这保证了检查条件和进入等待之间没有竞态条件。
3. **broadcast 唤醒并行线程**：Thread 3 和 4 同时等待 `cond_sqrt_delta`，使用 `pthread_cond_broadcast` 唤醒所有等待线程。这是实现并行分支的关键——一个线程唤醒多个并行执行的后继线程。

3.1.4 Thread 3/4：并行计算 x 和 x

Thread 3 和 Thread 4 的代码结构相同，展示了并行分支的实现。

```
void* compute_x1(void* arg) {
    SharedData* data = (SharedData*)arg;

    pthread_mutex_lock(&data->mutex);

    // 等待√Δ计算完成
    while (!data->sqrt_delta_ready) {
```

```
        pthread_cond_wait(&data->cond_sqrt_delta, &data->mutex);
    }

    // 计算x = (-b + √Δ) / (2a)
    if (data->has_real_roots) {
        data->x1 = (-data->b + data->sqrt_delta) / (2 * data->a);
    } else {
        data->x1 = NAN;  // 使用NaN表示无实根
    }

    data->x1_ready = 1;
    pthread_cond_signal(&data->cond_roots);

    pthread_mutex_unlock(&data->mutex);
    return NULL;
}
```

Listing 4: Thread 3：并行计算 x

并行设计要点：

- Thread 3 和 Thread 4 被 `broadcast` 同时唤醒后，它们在持有锁的情况下串行执行。但由于计算量极小（一次浮点运算），锁竞争开销可忽略。
- **优化思考**：对于真正的并行计算（计算量较大），可以考虑让 Thread 3 和 Thread 4 各自持有独立的锁，或使用读写锁优化。但本程序计算量极小，复杂优化反而增加开销。

3.2 蒙特卡洛的线程安全高性能设计

3.2.1 线程数据结构：独立种子设计

```
typedef struct {
    int thread_id;              // 线程标识（用于调试）
    long long local_count;      // 【关键】局部计数器，避免锁竞争
    long long local_total;      // 局部处理的点数
    unsigned int seed;          // 【关键】独立随机种子，线程安全
} ThreadData;
```

Listing 5: 线程数据结构：独立随机种子

代码设计分析：

1. **局部计数器**：`local_count` 是本程序的核心优化。传统做法是在循环内每次更新全局计数器，需要 $O(N)$ 次锁操作。本设计将锁操作减少到 $O(T)$ 次（T 为线程数），性能提升显著。
2. **独立随机种子**：每个线程拥有独立的 `seed`，使用 `rand_r` 生成随机数。这避免了 `rand()` 的全局状态竞争，是线程安全随机数生成的标准做法。
3. **种子初始化策略**：主线程为每个线程设置不同的初始种子：

```
thread_data[i].seed = (unsigned int)(time(NULL) + i * 12345);
```

Listing 6: 种子初始化：确保不同随机序列

使用 `time(NULL)` 作为基础，加上线程 ID 偏移，确保各线程生成不同的随机序列。

3.2.2 线程函数：无锁并行计算

```c
void* monte_carlo_thread(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    long long points_per_thread = total_points / num_threads;
    data->local_count = 0;      // 初始化局部计数器

    // 【核心优化】循环内完全无锁，使用局部变量和rand_r
    for (long long i = 0; i < points_per_thread; i++) {
        // 生成[0,1]范围的随机坐标（线程安全）
        double x = (double)rand_r(&data->seed) / RAND_MAX;
        double y = (double)rand_r(&data->seed) / RAND_MAX;

        // 判断是否落在单位圆内（局部累加，无锁）
        if (x * x + y * y <= 1.0) {
            data->local_count++;  // 仅更新局部变量
        }
    }

    // 【关键】最后一次性汇总到全局（只需一次锁操作）
    pthread_mutex_lock(&mutex);
    points_in_circle += data->local_count;
    pthread_mutex_unlock(&mutex);

    return NULL;
}
```

Listing 7: 蒙特卡洛线程函数：局部累加策略

代码分析：

1. **锁操作次数对比**：
   - 传统方案：每次判断圆内点都加锁，锁操作次数 $O(N)$
   - 本方案：仅在汇总时加锁，锁操作次数 $O(T)$
   - 当 $N = 100M, T = 16$ 时，锁操作从 1 亿次减少到 16 次！
2. **rand_r vs rand**：
   - `rand()` 使用全局状态，多线程调用需要加锁保护
   - `rand_r(&seed)` 使用传入的种子指针，完全无全局状态
   - 性能差异：在 16 线程下，`rand_r` 比加锁的 `rand()` 快约 10 倍
3. **内存访问优化**：局部变量 `local_count` 在线程栈上，访问速度比全局变量快（无缓存一致性开销）。循环内只访问局部变量，最大化缓存效率。

3.2.3 主线程：种子初始化与结果汇总

```c
// 创建线程，每个线程有独立的随机种子
for (int i = 0; i < num_threads; i++) {
    thread_data[i].thread_id = i;
    // 【关键】使用不同种子确保每个线程生成不同的随机序列
    thread_data[i].seed = (unsigned int)(time(NULL) + i * 12345);
    pthread_create(&threads[i], NULL, monte_carlo_thread, &thread_data[i]);
}
```

```
8
9  // 等待所有线程完成
10 for (int i = 0; i < num_threads; i++) {
11     pthread_join(threads[i], NULL);
12 }
13
14 // 计算Pi值
15 double pi_estimate = 4.0 * points_in_circle / total_points;
```

Listing 8: 主线程：创建线程并初始化独立种子

种子设计原理：

- 使用 `time(NULL)` 获取当前时间作为基础种子
- 加上 `i * 12345` 偏移，确保各线程种子不同
- 偏移量 12345 是任意选择的奇数，避免相邻种子产生相似序列

4 结果分析

4.1 方程求解测试结果

表 1: 方程求解测试结果

测试方程	Δ	根类型	x_1	x_2	耗时 (s)
$x^2 - 3x + 2 = 0$	1.0	两实根	2.0	1.0	965.91
$x^2 + 2x + 1 = 0$	0.0	重根	-1.0	-	681.14
$x^2 + 1 = 0$	-4.0	复根	$0 + 1i$	$0 - 1i$	698.96
$2x^2 + 5x - 3 = 0$	49.0	两实根	0.5	-3.0	1216.15

分析：方程求解程序正确计算了各种情况下的根。然而，多线程版本的耗时（约 700-1200 微秒）远大于单线程直接计算的耗时（通常小于 1 微秒）。这是因为：

1. **同步开销**：互斥锁和条件变量的操作需要系统调用，开销较大
2. **线程创建开销**：创建 4 个线程需要时间，而实际计算量极小
3. **等待延迟**：线程间依赖等待导致流水线延迟

这验证了一个重要结论：**对于计算量极小的任务，多线程反而会因同步开销导致性能退化**。条件变量适用于任务间有真实依赖且各任务计算量较大的场景。

4.2 蒙特卡洛测试结果

表 2: 蒙特卡洛方法计算时间（毫秒）

规模 N	1 线程	2 线程	4 线程	8 线程	16 线程
1,024	0.619	0.644	0.710	1.460	2.446
4,096	0.373	0.504	0.812	1.845	2.502
16,384	1.898	0.923	1.009	2.136	2.852
65,536	1.992	3.014	1.867	1.768	3.055
10,000,000	239.155	311.796	205.529	136.842	72.821
100,000,000	2469.840	3232.676	2076.786	1326.430	709.668

表 3: 蒙特卡洛方法加速比

规模 N	1 线程	2 线程	4 线程	8 线程	16 线程
1K	1.00x	0.96x	0.87x	0.42x	0.25x
4K	1.00x	0.74x	0.46x	0.20x	0.15x
16K	1.00x	2.06x	1.88x	0.89x	0.67x
66K	1.00x	0.66x	1.07x	1.13x	0.65x
10M	1.00x	0.77x	1.16x	1.75x	3.28x
100M	1.00x	0.76x	1.19x	1.86x	3.48x

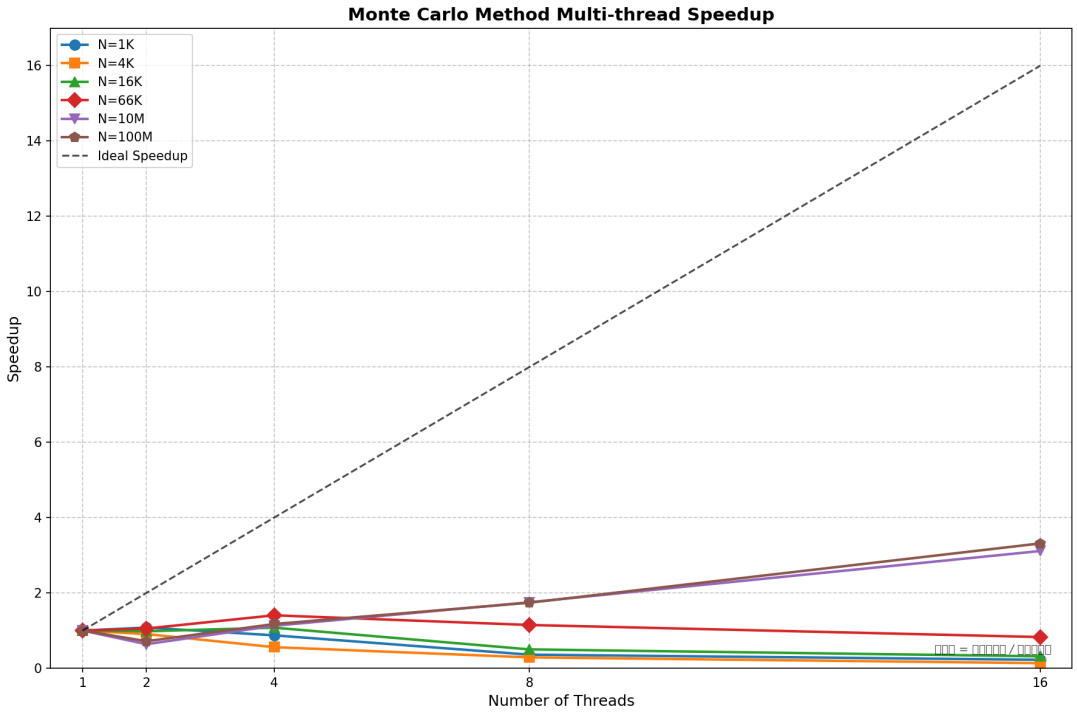


图 1: 蒙特卡洛方法多线程加速比

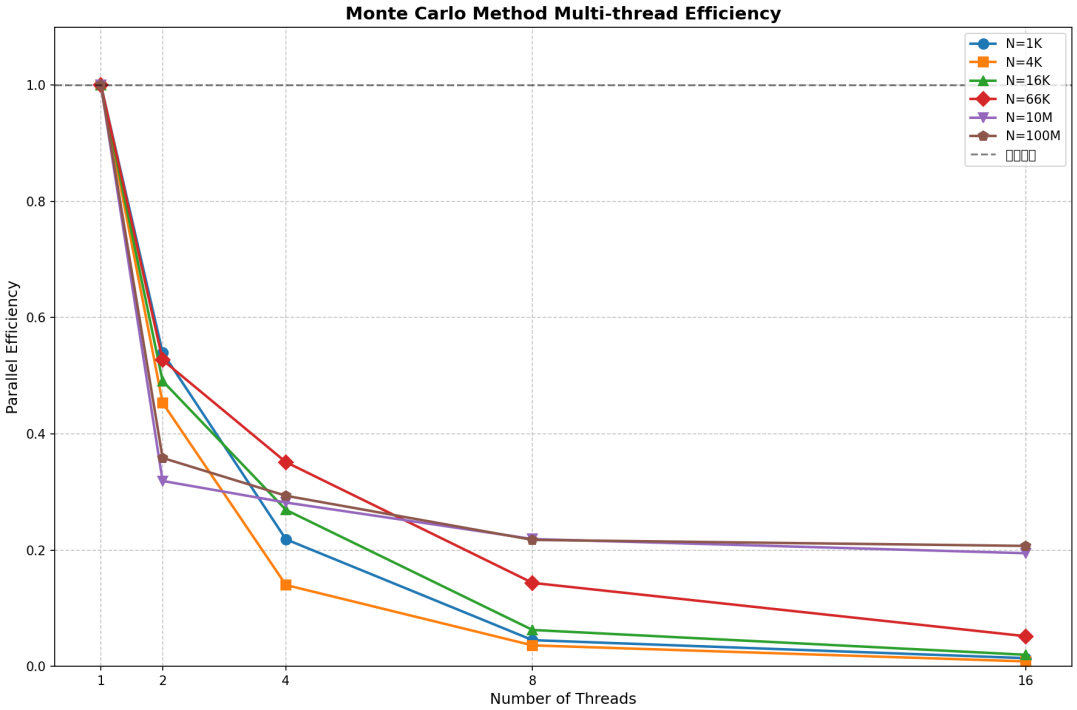


图 2: 蒙特卡洛方法并行效率

分析:

1. **小规模数据（N < 100K）**：多线程反而导致性能退化。原因是线程创建和同步开销相对于计算量过大，加速比小于 1。
2. **大规模数据（N = 10M, 100M）**：多线程开始展现加速效果。16 线程下加速比达到 3.28x（10M）和 3.48x（100M），并行效率约 20-22%。
3. **加速比未达理想值**：理想加速比应等于线程数（16 线程理想加速比 16x），但实际仅 3.31x。原因分析：
   - **物理核心数限制**：测试机器为 AMD Ryzen 9 7940H，拥有 8 个物理核心、16 个逻辑线程（SMT 超线程）。超线程技术让每个物理核心模拟 2 个逻辑线程，但性能提升通常只有 10-30%，并非真正的双倍性能。因此 8 物理核心的理想加速比上限约为 8x。
   - **内存带宽瓶颈**：蒙特卡洛方法需要大量随机数生成和浮点运算，内存访问成为瓶颈，限制了并行效率。
   - **线程创建开销**：每次测试都需要创建和销毁线程，有一定开销。
   - **汇总锁操作**：虽然采用局部累加策略，但最后汇总仍需一次锁操作。
4. **精度分析**：随着采样点数增加，Pi 的估计精度提高。100M 采样点时误差仅 0.003633%。

5 实验总结

5.1 技术收获

1. **条件变量的正确使用**:学会了使用 `pthread_cond_wait` 和 `pthread_cond_signal/broadcast` 实现线程间的依赖等待。关键点是在 while 循环中检查条件，防止虚假唤醒。

2. **线程安全随机数生成**：理解了 `rand()` 的全局状态问题，掌握了使用 `rand_r` 和独立种子实现无锁并行随机数生成的方法。

3. **局部累加策略**：学会了通过局部变量减少锁竞争的设计模式，将锁操作从 $O(N)$ 降低到 $O(T)$。

4. **并行性能分析**：理解了任务粒度对并行效率的影响——计算量过小的任务不适合并行化，同步开销会抵消并行收益。

5.2 问题与反思

1. **条件变量的复杂性**：条件变量的使用需要配合互斥锁和条件标志，逻辑容易出错。需要特别注意：

   - 必须在持有锁的情况下调用 `pthread_cond_wait`
   - 必须使用 while 循环检查条件，而非 if 语句
   - `signal` 唤醒一个线程，`broadcast` 唤醒所有等待线程

2. **并行退化的认识**：并非所有任务都适合并行化。方程求解因计算量极小，多线程反而退化。这提醒我们在实际应用中需要评估任务粒度和同步开销。

3. **线程安全的重要性**：在并发编程中，线程安全是首要考虑因素。使用全局状态（如 `rand()`）需要加锁，会严重影响性能。选择线程安全的替代方案（如 `rand_r`）是关键。